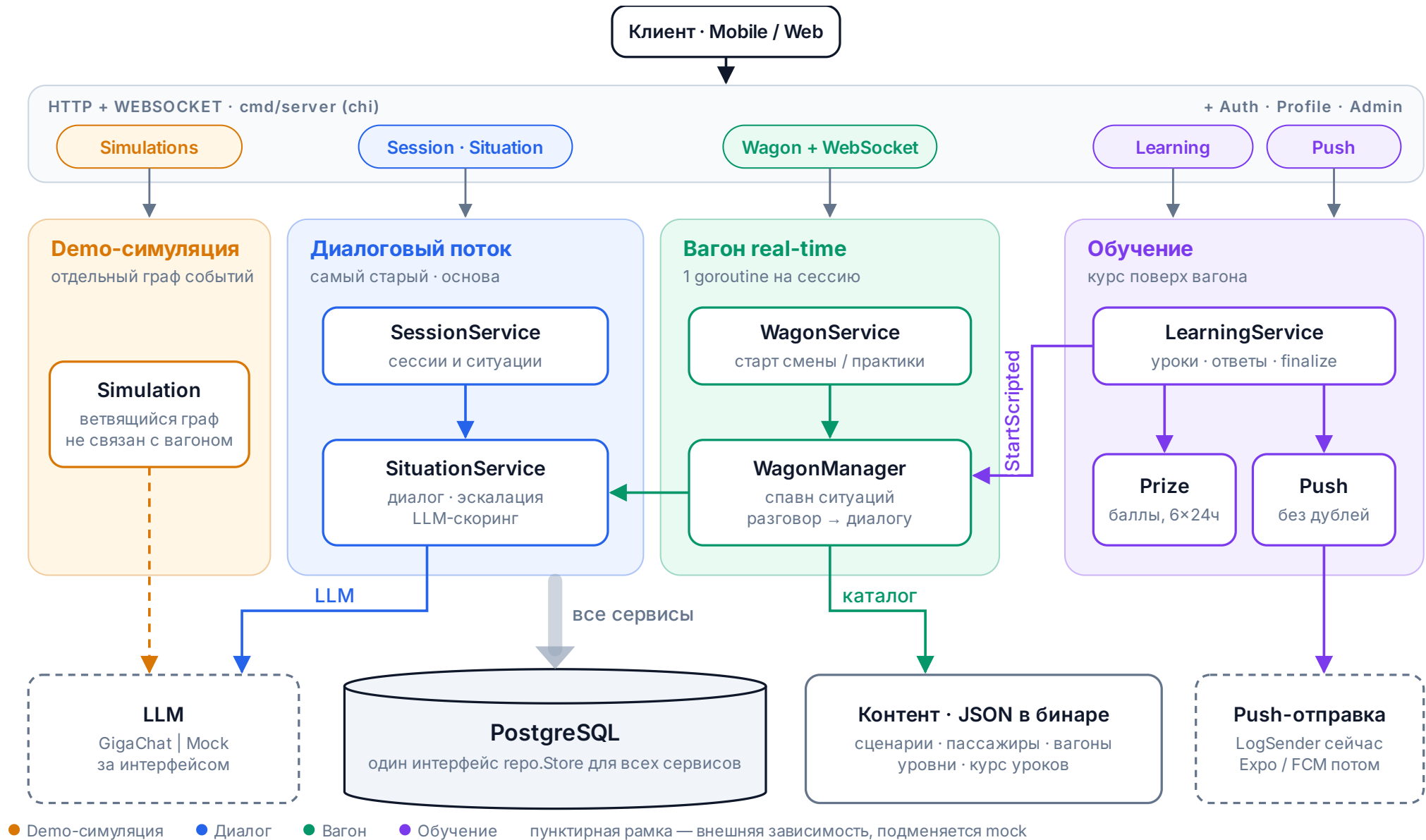


Диаграмма компонентов

Один Go-процесс: HTTP-обработчики → сервисы → PostgreSQL. Три игровых потока делят одни таблицы, но не код.
Обучение построено поверх вагона.



Как это устроено

Диалоговый поток

Самый старый. Ситуации из каталога, диалог с LLM, скоринг. Его переиспользует вагон для разговора с пассажиром.

Демо-симуляция

Отдельный ветвящийся граф событий. С вагоном никак не связан.

Вагон real-time

WebSocket, по одной goroutine на сессию. Решает, **когда и на ком** появится ситуация. Сам разговор отдаёт диалоговому потоку.

Обучение

Курс из глав и уроков. Запускает вагон по сценарию, после урока начисляет XP, бейдж, баллы и шлёт push.

Ключевые решения

Один Store на всё

Все сервисы зависят от интерфейса `repo.Store`, а не от конкретной БД. Реализация одна — Postgres.

Контент — это данные

Новый уровень, урок или сценарий — правка JSON, а не Go-кода. Ошибки в id ловятся при старте сервера.

Без двойных начислений

Уникальные ключи в БД + `ON CONFLICT DO NOTHING`. Один паттерн для XP, призовых баллов и push.

Внешнее — за интерфейсом

LLM (GigaChat) и push (Expo/FCM) подключаются одним новым файлом. По умолчанию стоит mock.